Memory Design and Data Handling Note

Status: Customer memory is persisted by the backend and loaded server-side into authenticated customer chat context. The internal console uses backend APIs for staff-owned memory and explicit, audited customer-memory support operations.

Purpose and use case

The memory feature stores a small set of customer preferences so a support reply can reflect the customer's preferred contact method and standing instructions without asking again in each conversation. These facts may inform wording and follow-up suggestions. They are not evidence of identity, consent for a particular transaction, or authority to perform an action.

The intended initial use case is deliberately narrow:

- preferred contact method (email, SMS, or phone call);
- an optional standing note supplied or edited by the customer.

The feature should not be used to store passwords, authentication secrets, payment-card data, government identifiers, health information, or unrelated third-party personal data. A standing note should contain only information needed to personalize support.

Current implementation and data inventory

The backend stores customer facts in `customer_memory`, keyed by `customer_id` and `memory_key`, with an `is_enabled` flag. The authenticated customer `/memory-facts` API supports listing, creating, updating, and deleting only the caller's own records (key and value validated; at most 50 facts per customer; owner always taken from the verified session). The customer app uses that API; chat loads only that customer's enabled facts when their `memory_enabled` master preference is on.

The internal console uses `/admin/memory` endpoints. Staff-owned notes are stored separately in `agent_memory_records` and are not loaded into customer chat. Staff can search for a specific customer, view that customer's facts, change the customer's memory master switch, and create, update, or delete an individual fact. Customer facts are not included in search results; the values are fetched only after the staff member explicitly opens a selected customer. Viewing and modifying customer memory writes an audit record to `customer_memory_access_logs`, recording actor, customer, action, and optional record ID but never the memory value.

The schema migration for these console tables is present in the workspace and must be applied to the target database before deploying the console integration.

The table below shows examples only, not a fixed or exhaustive schema. Customer and staff memory keys are data-driven.

Fact	Field key	Example	Use
Preferred contact method	`preferred_contact_method`	`email`	Personalize relevant status/follow-up replies.
Standing note	`standing_note`	“Please email me updates.”	Add customer-provided context to relevant replies.

Each customer fact also has an `enabled` flag. Disabled facts are omitted from server-built chat context. Disabling a fact does **not** erase its stored value.

Data flow and implementation boundary

1. The customer app loads saved facts from the authenticated `/memory-facts` endpoint. The backend derives the owner from the verified session.
2. Per-fact enable/disable state is persisted on the row; the master preference is persisted on `customer_profiles.memory_enabled`.
3. On each customer chat message, the backend loads only that caller's enabled facts, and only when the master preference is enabled.
4. The backend passes these facts to the model system prompt as untrusted, informational context. Values are bounded by `CUSTOMER_MEMORY_CONTEXT_BUDGET_CHARS` (default 1200 characters).
5. Memory is never accepted from the chat request body, never loaded for staff preview, and cannot authorize actions or override the approval gate.
6. The console obtains staff-owned and selected-customer records from protected backend routes; it does not use browser `localStorage` as the source of truth for these managed records.

The backend table is named `customer_memory` (not `customer_memory_facts`). Customer APIs derive ownership from the authenticated user and scope updates/deletes by both record ID and caller ID. Staff APIs require a staff role; customer records are fetched only after explicit selection, and read/change events are audit logged. Staff's own notes live in the separate `agent_memory_records` table, scoped to the signed-in staff account.

Ownership and access

For records accessed through the customer `/memory-facts` API and customer chat, the authenticated customer is the owner: reads are filtered by caller ID, and updates/deletes verify ownership. Chat memory is loaded server-side from that same caller ID. In the console, staff endpoints require an authenticated staff role, customer lookup requires an explicit search, and read/write actions for the selected customer's memory are logged. Staff personal notes are scoped to their own staff ID.

The intended production model is customer-owned, account-scoped data:

- The server must derive the owner from the authenticated session, never from a client-supplied customer ID.
- A customer may read, update, disable, and delete their own facts.
- Other customers must not be able to read or modify them.
- Staff access should be denied by default and granted only for a documented support need, with appropriate authorization and auditability.
- Customer facts must remain separate from staff-only operational memory.

Customer self-service (list, create, update, disable, delete) and authenticated staff support controls are implemented and covered by automated tests (`resolv-hq-backend/tests/memory.test.ts`). Retention, export and audit-read are now defined and implemented (see Retention and deletion); the backup window is still unverified.

Retention and deletion

Policy (defaults; configurable by environment variable)

Data	Retention	Deleted by
Customer memory facts	Until the customer deletes them,	Customer (one fact or

(customer_memory)	or **365 idle days** (MEMORY_RETENTION_DAYS) since the last change	all), staff for a selected customer (audited), or the retention purge
Staff notes (agent_memory_records)	Until the staff member deletes them or the account is removed (cascade). Not auto-purged	Owning staff member
Memory access audit (customer_memory_access_logs)	**730 days** (AUDIT_LOG_RETENTION_DAYS), then purged	Retention purge only; no API deletes audit rows
Chat conversations and messages	**365 idle days** (CONVERSATION_RETENTION_DAYS), only for conversations with no agent run attached	Retention purge; also removed if the customer account is deleted (cascade)
Agent runs, steps, tool executions, guardrail events, approvals	Kept at least 12 months and **never auto-deleted**: they are the audit trail for AI actions and human approval decisions. Review and purge manually under a documented procedure	Manual, admin

Setting a *_DAYS value to 0 disables that rule. The purge runs at startup and then daily **only when `RETENTION_ENABLED=true`**; it is off by default, so until it is switched on nothing expires automatically. An admin can preview what the purge would delete at GET /admin/memory/retention (dry run, counts only).

Changing a fact updates its updated_at, which resets its idle clock (this was fixed on 9 Oct; before then edits did not advance it).

Export and access

- A customer can download all of their saved facts as JSON from GET /memory-facts/export.
- An admin (not an agent) can read the memory audit trail at GET /admin/memory/audit (optional customerId, limit up to 200). It returns actor, customer, action, record ID and time, never fact values.
- Staff notes have no export. Chat transcripts have no export endpoint yet.

Deletion and backups

A deletion removes the row, and the next context assembly excludes it (verified by test). Deleted data may remain in database backups and point-in-time recovery copies until those age out. The backup window depends on the Supabase plan and was not verified here. Until it is confirmed and recorded, do not describe deletion as immediate everywhere.

Processing and safety boundaries

- Treat memory values as user-provided, untrusted data. They may contain mistakes or instructions that conflict with system policy.
- Use memory only as contextual information for a response; never interpret it as authorization or as a substitute for explicit, transaction-specific confirmation.
- Keep consequential actions behind the existing server-side approval workflow. Memory must not approve, submit, or bypass an action.
- Include only enabled, relevant, minimal facts in a model request. Avoid adding full chat history or unrelated profile fields to memory context.

- Do not place memory values in application logs, analytics, or error messages unless there is a documented need and appropriate access controls.
- The authenticated chat request may include client memory fields, but the backend does not trust or consume them; it independently loads the verified customer's enabled records.

Production-readiness work still required

Remaining work:

7. Turn the retention purge on (`RETENTION_ENABLED=true`) after the lead approves the default periods, and confirm the Supabase backup window.
8. Add tests for malicious instruction content inside a fact and for the approval gate with a memory-influenced draft. Cross-account isolation, staff authorization/audit behavior and disabled/master-off exclusion are covered by `tests/memory.test.ts`, and the create/update/export/loader/delete path ran against the real database on 9 Oct (`npm run test:db`).

Implementation references

- [../src/app/(console)/memory/memory-access-panel.tsx](https://file+.vscode-resource.vscode-cdn.net/c%3A/Users/Hillary/Desktop/RESOLV_HQ/resolv-hq/src/app/\(console\)/memory/memory-access-panel.tsx) — staff memory and selected-customer memory interface.
- [../src/lib/stores/chat-store.ts](https://file+.vscode-resource.vscode-cdn.net/c%3A/Users/Hillary/Desktop/RESOLV_HQ/resolv-hq/src/lib/stores/chat-store.ts) — customer-facing chat state and local fallback behavior.
- [../src/components/chat/memory-settings.tsx](https://file+.vscode-resource.vscode-cdn.net/c%3A/Users/Hillary/Desktop/RESOLV_HQ/resolv-hq/src/components/chat/memory-settings.tsx) — customer-facing controls.
- [../../resolv-hq-backend/src/routes/memory.ts](https://file+.vscode-resource.vscode-cdn.net/c%3A/Users/Hillary/Desktop/RESOLV_HQ/resolv-hq-backend/src/routes/memory.ts) — authenticated customer-scoped read/update/delete API.
- [../../resolv-hq-backend/src/routes/admin/memory.ts](https://file+.vscode-resource.vscode-cdn.net/c%3A/Users/Hillary/Desktop/RESOLV_HQ/resolv-hq-backend/src/routes/admin/memory.ts) — staff-only aggregate metadata, staff-owned CRUD, customer search and selected-customer management with audit logging.
- [../../resolv-hq-backend/src/routes/chat.ts](https://file+.vscode-resource.vscode-cdn.net/c%3A/Users/Hillary/Desktop/RESOLV_HQ/resolv-hq-backend/src/routes/chat.ts) — authenticated message handler that loads customer memory server-side.
- [../../resolv-hq-backend/prisma/migrations/20261008100000_console_memory_integration/migration.sql](https://file+.vscode-resource.vscode-cdn.net/c%3A/Users/Hillary/Desktop/RESOLV_HQ/resolv-hq-backend/prisma/migrations/20261008100000_console_memory_integration/migration.sql) — staff memory and customer memory access-audit tables.
- [../../resolv-hq-backend/src/routes/admin/tools.ts](https://file+.vscode-resource.vscode-cdn.net/c%3A/Users/Hillary/Desktop/RESOLV_HQ/resolv-hq-backend/src/routes/admin/tools.ts) — live tool registry response and input schemas.